

Neural-Swarm: Decentralized Close-Proximity Multirotor Control Using Learned Interactions

Guanya Shi, Wolfgang Hönig, Yisong Yue, and Soon-Jo Chung

Abstract—In this paper, we present *Neural-Swarm*, a non-linear decentralized stable controller for close-proximity flight of multirotor swarms. Close-proximity control is challenging due to the complex aerodynamic interaction effects between multirotors, such as downwash from higher vehicles to lower ones. Conventional methods often fail to properly capture these interaction effects, resulting in controllers that must maintain large safety distances between vehicles, and thus are not capable of close-proximity flight. Our approach combines a nominal dynamics model with a regularized permutation-invariant Deep Neural Network (DNN) that accurately learns the high-order multi-vehicle interactions. We design a stable nonlinear tracking controller using the learned model. Experimental results demonstrate that the proposed controller significantly outperforms a baseline nonlinear tracking controller with up to four times smaller worst-case height tracking errors. We also empirically demonstrate the ability of our learned model to generalize to larger swarm sizes.

I. INTRODUCTION

The ongoing commoditization of unmanned aerial vehicles (UAVs) is propelling interest in advanced control methods for large aerial swarms [1], [2]. Potential applications are plentiful, including manipulation, search, surveillance, mapping, amongst many others. Many settings require the UAVs to fly in close proximity to each other, also known as dense formation control. For example, consider a search-and-rescue mission where the aerial swarm must enter and search a collapsed building. In such scenarios, close-proximity flight enables the swarm to navigate the building much faster compared to swarms that must maintain large distances from each other.

A major challenge of close-proximity control is that the small distance between UAVs creates complex aerodynamic interactions. For instance, one multirotor flying above another causes the so-called downwash effect on the lower one, which is difficult to model using conventional approaches [3]. In lieu of better downwash interaction modeling, one must require a large safety distance between vehicles, e.g., 60 cm for the small Crazyflie 2.0 quadrotor (9 cm rotor-to-rotor) [4]. However, the downwash for two Crazyflie quadrotors hovering 30 cm on top of each other is only -9 g, which is well within their thrust capabilities, and suggests that proper modeling of downwash and other interaction effects can lead to more precise dense formation control.

Demo videos: <https://youtu.be/v4j-9pH1lQ8>

The authors are with California Institute of Technology, USA. {gshi, whoenig, yyue, sjchung}@caltech.edu.

We thank Michael O’Connell and Xichen Shi for helpful discussions and Anya Vinogradsky for help with the firmware implementation.

The work is funded in part by Caltech’s Center for Autonomous Systems and Technologies (CAST) and the Raytheon Company.

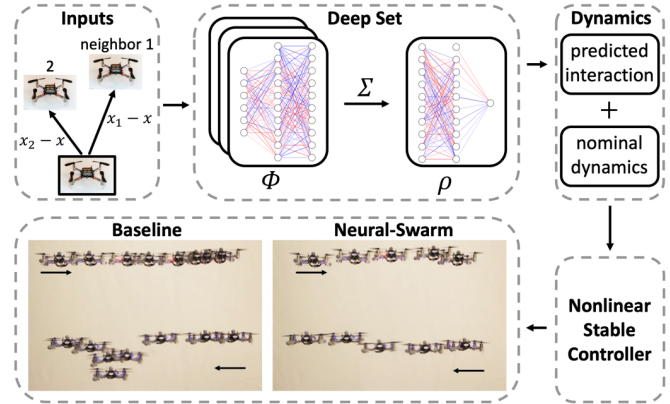


Fig. 1. We learn complex interaction between multirotors using regularized deep sets and design nonlinear stable controller for close-proximity flight.

In this paper, we propose a learning-based controller, *Neural-Swarm*, to improve the precision of close-proximity control of homogeneous multirotor swarms. In particular, we train a regularized permutation-invariant deep neural network (DNN) to predict the residual interaction forces not captured by nominal models of free-space aerodynamics. The DNN only requires relative positions and velocities of neighboring multirotors as inputs, similar to existing collision-avoidance techniques [5], which enables a fully decentralized computation. We use the predicted interaction forces as a feed-forward term in the multirotors’ position controller, which enables close-proximity flight. Our solution is computationally efficient and can run in real-time on a small 32-bit microcontroller. We validate our approach on different tasks using two to five quadrotors. To our knowledge, our approach is the first that models interactions between more than two multirotor vehicles.

From a learning perspective, we leverage two state-of-the-art tools to arrive at effective DNN models. The first is spectral normalization [6], which ensures the DNN is Lipschitz continuous. As in our prior work [7], Lipschitz continuity enables us to derive stability guarantees, and also helps the DNN generalize well on test examples that lie outside the training set. We also employ deep sets [8] to encode multi-vehicle interactions in an index-free or permutation-invariant manner, enabling better generalization to new formations and varying number of vehicles.

Related Work: The use of DNNs to learn higher-order residual dynamics or control outputs is becoming increasingly common across a range of control and reinforcement learning settings [7], [9]–[14]. The closest approach to ours is the Neural Lander [7], which uses a DNN to capture the

interaction between a single UAV and the ground, i.e., the well-studied ground effect [3], [15], [16]. In contrast, our work focuses on learning inter-vehicle aerodynamic interactions between several multirotors.

The interaction between two rotor blades has been studied in a lab setting to optimize the placement of rotors on a multirotor [17]. However, it remains an open question how this influences the flight of two or more multirotors in close proximity. Interactions between two multirotors can be estimated using a propeller velocity field model [18]. Unfortunately, this method is hard to generalize to the multi-robot case and this method only considers the stationary case, which will not work for many scenarios like swapping in Fig. 1. We instead use a learning-based method that can directly estimate the interaction forces of multiple neighboring robots from training data.

For motion planning, empirical models have been used to avoid harmful interactions [2], [19]–[21]. Typical safe interaction shapes are ellipsoids or cylinders and such models work for homogeneous and heterogeneous multirotor teams. Estimating such shapes requires potentially dangerous flight tests and the shapes are in general conservative. In contrast, we use learning to estimate the interaction forces accurately and use those forces in the controller to improve trajectory tracking performance in close-proximity flight. The learned forces can potentially be used for motion planning as well.

II. PROBLEM STATEMENT: SWARM INTERACTIONS

A. Single Multirotor Dynamics

A single multirotor's state comprises of the global position $\mathbf{p} \in \mathbb{R}^3$, global velocity $\mathbf{v} \in \mathbb{R}^3$, attitude rotation matrix $R \in \text{SO}(3)$, and body angular velocity $\boldsymbol{\omega} \in \mathbb{R}^3$. We consider the following dynamics:

$$\dot{\mathbf{p}} = \mathbf{v}, \quad m\dot{\mathbf{v}} = m\mathbf{g} + R\mathbf{f}_u + \mathbf{f}_a, \quad (1a)$$

$$\dot{R} = RS(\boldsymbol{\omega}), \quad J\dot{\boldsymbol{\omega}} = J\boldsymbol{\omega} \times \boldsymbol{\omega} + \boldsymbol{\tau}_u + \boldsymbol{\tau}_a, \quad (1b)$$

where m and J are the mass and inertia matrix of the system, respectively; $S(\cdot)$ is a skew-symmetric mapping; $\mathbf{g} = [0, 0, -g]^\top$ is the gravity vector; and $\mathbf{f}_u = [0, 0, T]^\top$ and $\boldsymbol{\tau}_u = [\tau_x, \tau_y, \tau_z]^\top$ are the total thrust and body torques from the rotors, respectively. The output wrench $\boldsymbol{\eta} = [T, \tau_x, \tau_y, \tau_z]^\top$ is linearly related to the control input $\boldsymbol{\eta} = B_0\mathbf{u}$, where $\mathbf{u} = [n_1^2, n_2^2, \dots, n_k^2]^\top$ is the squared motor speeds for a vehicle with k rotors and B_0 is the actuation matrix. The key difficulty stems from disturbance forces $\mathbf{f}_a = [f_{a,x}, f_{a,y}, f_{a,z}]^\top$ and disturbance torques $\boldsymbol{\tau}_a = [\tau_{a,x}, \tau_{a,y}, \tau_{a,z}]^\top$, generated by other multirotors.

B. Swarm Dynamics

Consider n homogeneous multirotors. To simplify notations, we use $\mathbf{x}^{(i)} = [\mathbf{p}^{(i)}; \mathbf{v}^{(i)}; R^{(i)}; \boldsymbol{\omega}^{(i)}]$ to denote the state of the i^{th} multirotor. Then (1) can be simplified as:

$$\dot{\mathbf{x}}^{(i)} = f(\mathbf{x}^{(i)}, \mathbf{u}^{(i)}) + \begin{bmatrix} \mathbf{0} \\ \mathbf{f}_a^{(i)} \\ \mathbf{0} \\ \boldsymbol{\tau}_a^{(i)} \end{bmatrix}, \quad i = 1, \dots, n, \quad (2)$$

where $f(\mathbf{x}^{(i)}, \mathbf{u}^{(i)})$ is the nominal dynamics and $\mathbf{f}_a^{(i)}$ and $\boldsymbol{\tau}_a^{(i)}$ are unmodeled force and torque from interactions between other multirotors.

We use $\mathbf{x}^{(ij)}$ to denote the relative state component between robot i and j , e.g., $\mathbf{x}^{(ij)} = [\mathbf{p}^{(j)} - \mathbf{p}^{(i)}; \mathbf{v}^{(j)} - \mathbf{v}^{(i)}]$. For robot i , the unmodeled force and torque in (2) are functions of relative states to its neighbors,

$$\mathbf{f}_a^{(i)} = \mathbf{f}_a(\mathcal{N}^{(i)}) \text{ and } \boldsymbol{\tau}_a^{(i)} = \boldsymbol{\tau}_a(\mathcal{N}^{(i)}), \quad (3)$$

where $\mathcal{N}^{(i)} = \{\mathbf{x}^{(ij)} | j \in \text{neighbor}(i)\}$ is the set of the relative states of the neighbors of i . Note that here we assume the swarm system is homogeneous, i.e., each robot has the same functions f , $\mathbf{f}_a$, and $\boldsymbol{\tau}_a$.

C. Problem Statement & Approach

We aim to improve the control performance of a multirotor swarm during close formation flight, by learning the unknown interaction terms $\mathbf{f}_a$ and $\boldsymbol{\tau}_a$. Here, we focus on the position dynamics (1a) so $\mathbf{f}_a$ is our primary concern.

We first approximate $\mathbf{f}_a$ using a permutation invariant deep neural network (DNN), and then incorporate the DNN in our exponentially-stabilizing controller. Training is done offline, and the learned interaction dynamics model is applied in the on-board controller in real-time.

III. LEARNING APPROACH

We employ state-of-the-art deep learning methods to capture the unknown (or residual) multi-vehicle interaction effects. In particular, we require that the deep neural nets (DNNs) have strong Lipschitz properties (for stability analysis), can generalize well to new test cases, and use compact encodings to achieve high computational and statistical efficiency. To that end, we employ deep sets [8] and spectral normalization [6] in conjunction with a standard feed-forward neural architecture.¹

A. Permutation-Invariant Neural Networks

The permutation-invariant aspect of the interaction term $\mathbf{f}_a$ (3) can be characterized as:

$$\mathbf{f}_a(\mathcal{N}^{(i)}) = \mathbf{f}_a(\pi(\mathcal{N}^{(i)})), \quad (4)$$

for any permutation π . Since our goal is to learn the function $\mathbf{f}_a$ using DNNs, we need to guarantee that the learned DNN is permutation-invariant. The following lemma (a corollary of Theorem 7 in [8]) gives the necessary and sufficient condition for a DNN to be permutation-invariant.

Lemma 1 (adapted from Thm 7 in [8]): A continuous function $h(\{\mathbf{x}_1, \dots, \mathbf{x}_k\})$, with $\mathbf{x}_i \in [\mathbf{x}_{\min}, \mathbf{x}_{\max}]^n$, is permutation-invariant if and only if it is decomposable into $\rho(\sum_{i=1}^k \phi(\mathbf{x}_i))$, for some functions ϕ and ρ .

The proof from [8] is highly non-trivial and only holds for a fixed number of vehicles k . Furthermore, their proof

¹An alternative approach is to discretize the input space and employ convolutional neural networks (CNNs), which also yields a permutation-invariant encoding. However, CNNs suffer from two limitations: 1) they require much more training data and computation; and 2) they are restricted to a pre-determined resolution and input domain.

technique (which is likely loose) involves a large expansion in the intrinsic dimensionality (specifically ρ) compared to the dimensionality of h . We will show in our experiments that ρ and ϕ can be learned using relatively compact DNNs, and can generalize well to larger swarms.

Lemma 1 implies we can consider the following “deep sets” [8] architecture to approximate $\mathbf{f}_a$:

$$\mathbf{f}_a^{(i)} = \mathbf{f}_a(\mathcal{N}^{(i)}) \approx \rho \left(\sum_{\mathbf{x}^{(ij)} \in \mathcal{N}^{(i)}} \phi(\mathbf{x}^{(ij)}, \boldsymbol{\theta}_\phi), \boldsymbol{\theta}_\rho \right) = \hat{\mathbf{f}}_a^{(i)}, \quad (5)$$

where $\phi(\cdot)$ and $\rho(\cdot)$ are two DNNs, and $\boldsymbol{\theta}_\phi$ and $\boldsymbol{\theta}_\rho$ are their corresponding parameters. The output of ϕ is a hidden state to represent “contributions” from each neighbor, and ρ is a nonlinear mapping from the summation of these hidden states to the total effect. The advantages of this approach are:

- **Representation ability.** Since Lemma 1 is necessary and sufficient, we do not lose approximation power by using this constrained framework. We demonstrate strong empirical performance using relatively compact DNNs for ρ and ϕ .
- **Computational and sampling efficiency and scalability.** Since the input dimension of $\phi(\cdot)$ is always the same as the single vehicle case, the feed-forward computational complexity of (5) grows linearly with the number of neighboring vehicles. Moreover, given training data from n vehicles, under the homogeneous dynamics assumption, we can reuse the data n times. In practice, we found that a few minutes flight data is sufficient to accurately learn interactions between two to five multirotors.
- **Generalization to varying swarm size.** Given learned $\phi(\cdot)$ and $\rho(\cdot)$, (5) can be used to predict interactions for any swarm size. In other words, a model trained on swarms of a certain size may also accurately model (slightly) larger swarms. In practice, we found that trained with data from three multirotor swarms, our model can give good predictions for five multirotor swarms.

B. Spectral Normalization for Robustness and Generalization

To improve robustness and generalization of DNNs, we use spectral normalization [6] for training optimization. Spectral normalization stabilizes DNN training by constraining its Lipschitz constant. Spectrally normalized DNNs have been shown to generalize well, which is an indication of stability in machine learning. Spectrally normalized DNNs have also been shown to be robust, which can be used to provide control-theoretic stability guarantees [7], [22].

Mathematically, the Lipschitz constant of a function $\|g\|_{\text{Lip}}$ is defined as the smallest value such that:

$$\forall \mathbf{x}, \mathbf{x}' : \|g(\mathbf{x}) - g(\mathbf{x}')\|_2 / \|\mathbf{x} - \mathbf{x}'\|_2 \leq \|g\|_{\text{Lip}}.$$

Let $g(\mathbf{x}, \boldsymbol{\theta})$ be a ReLU DNN parameterized by the DNN weights $\boldsymbol{\theta} = W^1, \dots, W^{L+1}$:

$$g(\mathbf{x}, \boldsymbol{\theta}) = W^{L+1} \sigma(W^L \sigma(\dots \sigma(W^1 \mathbf{x}) \dots)), \quad (6)$$

where the activation function $\sigma(\cdot) = \max(\cdot, 0)$ is called the element-wise ReLU function. In practice, we apply the spectral normalization to the weight matrices in each layer after each batch gradient descent as follows:

$$W^i \leftarrow W^i / \|W^i\|_2 \cdot \gamma^{\frac{1}{L+1}}, i = 1, \dots, L+1, \quad (7)$$

where $\|W^i\|_2$ is the maximum singular value of W^i and γ is a hyperparameter. With (7), $\|g\|_{\text{Lip}}$ will be upper bounded by γ . Since spectrally normalized g is γ -Lipschitz continuous, it is robust to noise $\Delta \mathbf{x}$, i.e., $\|g(\mathbf{x} + \Delta \mathbf{x}) - g(\mathbf{x})\|_2$ is always bounded by $\gamma \|\Delta \mathbf{x}\|_2$. In this paper, we apply the spectral normalization on both the $\phi(\cdot)$ and $\rho(\cdot)$ DNNs in (5).

C. Data Collection

Learning a DNN to approximate $\mathbf{f}_a$ requires collecting close formation flight data. However, the downwash effect causes the nominally controlled multirotors (without compensation for the interaction forces) to move apart from each other, see Fig. 1. Thus, we use a cumulative/curriculum learning approach: first, we collect data for two multirotors without a DNN and learn a model. Second, we repeat the data collection using our learned model as feed-forward term, which allows closer-proximity flight of the two vehicle. Third, we repeat the procedure with increasing number of vehicles, using the current best model.

Note that our data collection and learning are independent of the controller used and independent of the $\mathbf{f}_a$ compensation. In particular, if we actively compensate for a learned $\hat{\mathbf{f}}_a$, this will only affect $\mathbf{f}_u$ in (1a) and not the observed $\mathbf{f}_a$.

IV. NONLINEAR DECENTRALIZED CONTROLLER DESIGN

Our *Neural-Swarm* controller is a nonlinear feedback linearization controller using the learned interaction term $\hat{\mathbf{f}}_a^{(i)}$. Note that *Neural-Swarm* is decentralized, since $\hat{\mathbf{f}}_a^{(i)}$ is a function of the neighbor set, $\mathcal{N}^{(i)}$, of vehicle i . Moreover, the computational complexity of $\hat{\mathbf{f}}_a^{(i)}(\mathcal{N}^{(i)})$ grows linearly as the size of $\mathcal{N}^{(i)}$, since we employ deep sets to encode $\hat{\mathbf{f}}_a^{(i)}$.

A. Reference Trajectory Tracking

Similar to [7], we employ an integral controller that accounts for the predicted residual dynamics, which in our case are the multi-vehicle interaction effects. For vehicle i , we define the position tracking error as $\tilde{\mathbf{p}}^{(i)} = \mathbf{p}^{(i)} - \mathbf{p}_d^{(i)}$ and the composite variable $\mathbf{s}^{(i)}$ as:

$$\mathbf{s}^{(i)} = \dot{\tilde{\mathbf{p}}}^{(i)} + 2\Lambda \tilde{\mathbf{p}}^{(i)} + \Lambda^2 \int \tilde{\mathbf{p}}^{(i)} dt = \dot{\mathbf{p}}^{(i)} - \mathbf{v}_r^{(i)}, \quad (8)$$

where $\mathbf{v}_r^{(i)} = \dot{\mathbf{p}}_d^{(i)} - 2\Lambda \tilde{\mathbf{p}}^{(i)} - \Lambda^2 \int \tilde{\mathbf{p}}^{(i)} dt$ is the reference velocity. We design the total desired rotor force $\mathbf{f}_d^{(i)}$ as:

$$\begin{aligned} \mathbf{f}_d^{(i)} &= m \dot{\mathbf{v}}_r^{(i)} - K \mathbf{s}^{(i)} - m \mathbf{g} - \hat{\mathbf{f}}_a^{(i)}, \\ \hat{\mathbf{f}}_a^{(i)} &= \rho \left(\sum_{\mathbf{x}^{(ij)} \in \mathcal{N}^{(i)}} \phi(\mathbf{x}^{(ij)}, \boldsymbol{\theta}_\phi), \boldsymbol{\theta}_\rho \right). \end{aligned} \quad (9)$$

Note that the position control law in (9) is decentralized, because we only consider the relative states $\mathbf{x}^{(ij)} \in \mathcal{N}^{(i)}$ in the controller.

Using $\mathbf{f}_d^{(i)}$, the desired total thrust $T_d^{(i)}$ and desired attitude $R_d^{(i)}$ can be easily computed [23]. Given $R_d^{(i)}$, we can use any attitude controller to compute $\tau_d^{(i)}$, for example robust nonlinear tracking control with global exponential stability [23], or geometric tracking control on SE(3) [24]. From this process, we get $\eta_d^{(i)} = [T_d^{(i)}; \tau_d^{(i)}]$, and then the desired control signal of each vehicle is $\mathbf{u}_d^{(i)} = B_0^\dagger \eta_d^{(i)}$, which can be computed in a decentralized manner for each vehicle.

B. Nonlinear Stability and Robustness Analysis

Note that since $\|\hat{\mathbf{f}}_a^{(i)} - \mathbf{f}_a^{(i)}\| \neq 0$, we can not guarantee the tracking error $\tilde{\mathbf{p}}^{(i)} \rightarrow 0$. However, under some mild assumptions, we can guarantee input-to-state stability (ISS) using exponential stability [25] for all the vehicles.

Assumption 1: The desired position trajectory $\mathbf{p}_d^{(i)}, \dot{\mathbf{p}}_d^{(i)}$, and $\ddot{\mathbf{p}}_d^{(i)}$ are bounded for all i .

Assumption 2: Define the learning error as $\hat{\mathbf{f}}_a^{(i)} - \mathbf{f}_a^{(i)}$, with two components: $\hat{\mathbf{f}}_a^{(i)} - \mathbf{f}_a^{(i)} = \epsilon_0^{(i)} + \epsilon^{(i)}(t)$, where $\epsilon_0^{(i)}$ is some constant bias and $\epsilon^{(i)}(t)$ is a time-varying term. We assume that for vehicle i , $\|\epsilon^{(i)}(t)\|$ is upper bounded by $d_m^{(i)}$.

Theorem 2: Under Assumptions 1 and 2, for vehicle i , for some desired trajectory $\mathbf{p}_d^{(i)}(t)$, (9) achieves exponential convergence of the tracking error to an error ball:

$$\lim_{t \rightarrow \infty} \|\tilde{\mathbf{p}}^{(i)}(t)\| = \frac{d_m^{(i)}}{\lambda_{\min}^2(\Lambda) \lambda_{\min}(K)}. \quad (10)$$

Proof: For vehicle i , consider the Lyapunov function $V^{(i)} = \frac{1}{2} m \|\dot{\mathbf{s}}^{(i)}\|^2$. With controller (9), we get the time-derivative of V :

$$\dot{V}^{(i)} = \dot{\mathbf{s}}^{(i)\top} (-K\dot{\mathbf{s}}^{(i)} + \dot{\epsilon}^{(i)}) \leq -\dot{\mathbf{s}}^{(i)\top} K\dot{\mathbf{s}}^{(i)} + \|\dot{\epsilon}^{(i)}\| d_m^{(i)}.$$

Using $V^{(i)} = \frac{1}{2} m \|\dot{\mathbf{s}}^{(i)}\|^2$, we have

$$\dot{V}^{(i)} \leq -\lambda_{\min}(K) \frac{2V^{(i)}}{m} + \sqrt{\frac{2V^{(i)}}{m}} d_m^{(i)}. \quad (11)$$

Using the Comparison Lemma [26], we obtain

$$\|\dot{\mathbf{s}}^{(i)}(t)\| \leq \|\dot{\mathbf{s}}^{(i)}(0)\| \exp\left(-\frac{\lambda_{\min}(K)}{m} \cdot t\right) + \frac{d_m^{(i)}}{\lambda_{\min}(K)}.$$

Note that $\dot{\mathbf{s}}^{(i)} = \ddot{\mathbf{p}}^{(i)} + 2\Lambda\dot{\tilde{\mathbf{p}}}^{(i)} + \Lambda^2\tilde{\mathbf{p}}^{(i)}$, and the hierarchical combination between $\dot{\mathbf{s}}^{(i)}$ and $\tilde{\mathbf{p}}^{(i)}$ results in $\lim_{t \rightarrow \infty} \|\tilde{\mathbf{p}}^{(i)}\| = \lim_{t \rightarrow \infty} \|\dot{\mathbf{s}}^{(i)}\| / \lambda_{\min}^2(\Lambda)$, yielding (10). ■

V. EXPERIMENTS

We use a slightly modified Crazyflie 2.0 (CF) as our quadrotor platform, a small (9 cm rotor-to-rotor) and lightweight (34 g) product that is commercially available. We use the Crazyswarm [27] package to control multiple Crazyflies simultaneously. Each quadrotor is equipped with four reflective markers for pose tracking at 100 Hz using a motion capture system. The nonlinear controller, extended Kalman filter, and neural network evaluation are running on-board the STM32 microcontroller.

For data collection, we use the uSD card extension board and store binary encoded data roughly every 10 ms. Each dataset is timestamped using the on-board microsecond timer

and the clocks are synchronized before takeoff using broadcast radio packets. The drift of the clocks of different Crazyflies can be ignored for our short flight times (less than 2 min).

A. Calibration and System Identification

Prior to learning the residual term $\mathbf{f}_a$, we first calibrate the nominal dynamics model $f(\mathbf{x}, \mathbf{u})$. We found that existing motor thrust models [28], [29] are not very accurate, because they only consider a single motor and ignore the effect of the battery state of charge. We calibrate each Crazyflie by mounting the whole quadrotor on a 100 g load cell which is directly connected to a custom extension board. We collect the current battery voltage, PWM signals (identical for all 4 motors), and measured force from the load cell for various motor speeds. We use this data to find two polynomial functions. The first computes the PWM signal given the current battery voltage and desired force. The second computes the maximum achievable force, given the current battery voltage. This second function is important for thrust mixing when motors are saturated [30].

We notice that the default motors and propellers can only produce a total force of about 48 g with a full battery, resulting in a best-case thrust-to-weight ratio of 1.4. Thus, we replaced the motors with more powerful ones (that have the same physical dimensions) to improve the best-case thrust-to-weight ratio to 2.6. We use the remaining parameters (J , thrust-to-torque ratio) from the existing literature [29].

B. Data Collection and Learning

We utilize two types data collection tasks: random walk and swapping. For random walk, we implement a simple reactive collision avoidance approach based on artificial potentials on-board each Crazyflie [31]. The host computer randomly selects new goal points within a small cube for each vehicle in a fixed frequency. Those goal points are used as an attractive force, while neighboring vehicles contribute a repulsive force. For swapping, we place vehicles in different horizontal planes on a cylinder and let them move to the opposite side. All vehicles are vertically aligned for one time instance, causing a large interaction force, see Fig. 1, 2, and 4 for examples with two, three, and four vehicles. The random walk data helps us to explore the whole space quickly, while the swapping data ensures that we have data for a specific task of interest. For both task types, we varied the scenarios from two to four vehicles, and collected one minute of data for each scenario.

To learn the interaction function $\mathbf{f}_a(\mathcal{N}^{(i)})$, we collect the timestamped states $\mathbf{x}^{(i)} = [\mathbf{p}^{(i)}; \mathbf{v}^{(i)}; \dot{\mathbf{v}}^{(i)}; R^{(i)}; \mathbf{f}_u^{(i)}]$ for each vehicle i . We then compute $\mathbf{y}^{(i)}$ as the observed value of $\mathbf{f}_a(\mathcal{N}^{(i)})$. We compute $\mathbf{f}_a^{(i)} = \mathbf{f}_a(\mathcal{N}^{(i)})$ using $m\dot{\mathbf{v}}^{(i)} = m\mathbf{g} + R^{(i)}\mathbf{f}_u^{(i)} + \mathbf{f}_a^{(i)}$ in (1a), where $\mathbf{f}_u^{(i)}$ is calculated based on our system identification in Sec. V-A. Our training data consists of sequences of $(\mathcal{N}^{(i)}, \mathbf{y}^{(i)})$ pairs, where $\mathcal{N}^{(i)} = \{\mathbf{x}^{(ij)} | j \in \text{neighbor}(i)\}$ is the set of the relative states of the neighbors of i . In practice, we compute the relative states from our collected data as $\mathbf{x}^{(ij)} = [\mathbf{p}^{(j)} - \mathbf{p}^{(i)}; \mathbf{v}^{(j)} - \mathbf{v}^{(i)}] \in \mathbb{R}^6$ (i.e., relative global position and relative global velocity), since the attitude information R and ω are not dominant for $\mathbf{f}_a$.

TABLE I

MAXIMUM z -ERROR (IN METERS) FOR VARYING SWARM SIZE SWAPPING TASKS AND NEURAL NETWORKS. TRAINING ON MORE VEHICLES LEADS TO THE BEST OVERALL PERFORMANCE FOR ALL SWARM SIZES.

Controller	Flight test	2 CF	3 CF	4 CF	5 CF
		Swap	Swap	Swap	Swap
Baseline		0.094	0.139	0.209	0.314
Trained w/ 2 CF		0.027	0.150	0.294	N.A.
Trained w/ 3 CF		0.026	0.082	0.140	0.159
Trained w/ 4 CF		0.024	0.061	0.102	0.150

In this work, we only learn the z component of $\mathbf{f}_a$ since we found the other two components, x and y , are very small, and do not significantly alter the nominal dynamics.

Since our swarm is homogeneous, each vehicle has the same function $\mathbf{f}_a$. Thus, we stack all the vehicle's data and train on them together, which implies more training data overall for larger swarms. Let $\mathcal{D}^{(i)}$ denote the training data of vehicle i , where the input-output pair is $(\mathcal{N}^{(i)}, \mathbf{y}^{(i)})$. We use the ReLU network class for both ϕ and ρ neural networks and our training loss is:

$$\sum_i \sum_{\mathcal{D}^{(i)}} \left\| \rho \left(\sum_{\mathbf{x}^{(ij)} \in \mathcal{N}^{(i)}} \phi(\mathbf{x}^{(ij)}, \boldsymbol{\theta}_\phi, \boldsymbol{\theta}_\rho) \right) - \mathbf{y}^{(i)} \right\|_2^2, \quad (12)$$

where $\boldsymbol{\theta}_\phi$ and $\boldsymbol{\theta}_\rho$ are neural network weights to be learned. Our ϕ DNN has four layers with architecture $6 \rightarrow 25 \rightarrow 40 \rightarrow 40 \rightarrow 40$, and our ρ DNN also has four layers, with architecture $40 \rightarrow 40 \rightarrow 40 \rightarrow 40 \rightarrow 1$. We use PyTorch [32] for training and implementation of spectral normalization (see Sec. III-B) of ϕ and ρ . We found that spectral normalization is in particular important for the small Crazyflie quadrotors, because their IMUs are directly mounted on the PCB frame causing more noisy measurements compared to bigger quadrotors.

Using the learned weights $\boldsymbol{\theta}_\phi$ and $\boldsymbol{\theta}_\rho$, we generate C-code to evaluate both networks efficiently on-board the quadrotor, similar to prior work [33]. The STM32 168 MHz microcontroller can evaluate each of the networks in about 550 μ s. Thus, we can compute $\mathbf{f}_a$ in less than 4 ms for 6 or less neighbors, which is sufficient for real-time operations.

C. Neural-Swarm Control Performance

We study the performance and generalization of different controllers on a swapping task using varying number of quadrotors. An example of our swapping task for two vehicles is shown in Fig. 1. The swapping task for multiple vehicles causes them to align vertically at one point in time with vertical distances of 0.2 m to 0.25 m between neighbors. This task is challenging, because: i) the lower vehicles experience downwash from multiple vehicles flying above; ii) the different velocity vectors of each vehicle creates interesting effects, including an effect where $\mathbf{f}_a$ is positive for a short period of time (see Fig. 3(b) for an example); and iii) for the case with more than two vehicles, the aerodynamic effect is not a simple superposition of each pair (see Fig. 3(c-f) for examples).

We use the following four controllers: 1) The baseline controller uses our position tracking controller (9) with

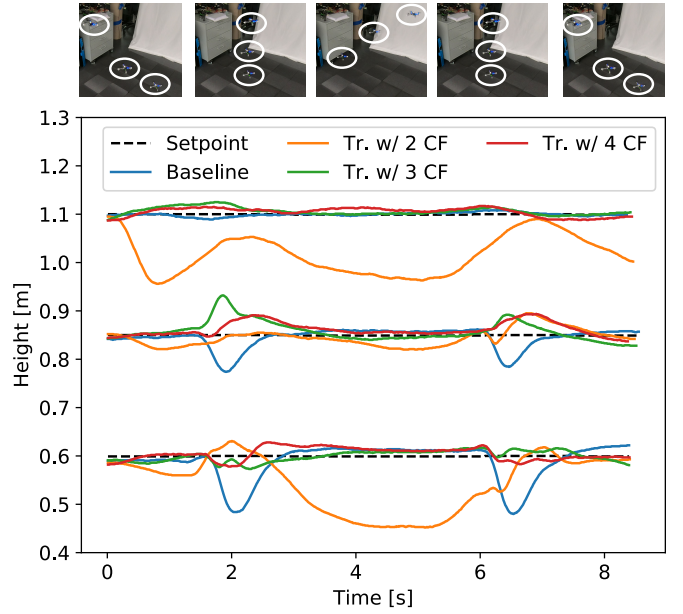


Fig. 2. Three vehicles moving using different control policies (corresponding to Table I, column 2). Each quadrotor flies at a different fixed height (25 cm vertical separation) and swaps sides such that all vehicles align vertically at $t = 2$ s and $t = 6.5$ s. Our approach trained on 3 or 4 vehicles controls the height much better than the baseline approach.

$\hat{\mathbf{f}}_a^{(i)} \equiv 0, \forall i$ and a nonlinear attitude tracking controller [24]; 2) – 4) The same controller with the same gains, but $\hat{\mathbf{f}}_a^{(i)}$ computed using different neural networks (trained on data flying 2, 3, and 4 quadrotors, respectively.) Note that all controllers, including the baseline controller, always have integral control compensation parts. Though an integral gain can cancel steady-state error during set-point regulation, it can struggle with complex time-variant interactions between vehicles. This issue is also reflected in the tracking error bound in Theorem 2. In Theorem 2, the tracking error will converge to $d_m^{(i)} / \lambda_{\min}^2(\Lambda) \lambda_{\min}(K)$. For our baseline we have $d_m^{(i)} = \sup_t \left\| \frac{d}{dt} \mathbf{f}_a^{(i)}(t) \right\|$, which means if $\mathbf{f}_a^{(i)}$ is changing fast as in the swapping task, our baseline will not perform well.

We repeat the swapping task for each controller six times, and report the maximum z -error that occurred for any vehicle over the whole flight. We also verified that the x - and y -error distributions are similar across the different controllers and do not report those numbers for brevity.

Results. Our results, described in Table I, show three important results: i) our controller successfully reduces the worst-case z -error by a factor of two to four (e.g., 2.4 cm instead of 9.4 cm for the two vehicle case); ii) our controller successfully generalizes to cases with more vehicles when trained with at least three vehicles (e.g., the controller trained with three quadrotors significantly improves flight performance even when flying five quadrotors); and iii) our controllers do not marginalize small-vehicle cases (e.g., the controller trained with four quadrotors works very well for the two-vehicle case). The observed maximum z -error for the test cases with three to five quadrotors is larger compared to the two-vehicle case because we occasionally saturate the motors during flight.

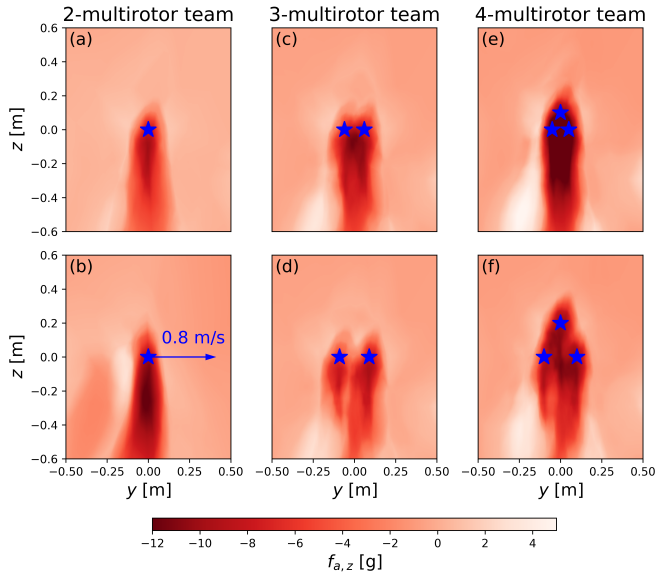


Fig. 3. $\hat{f}_{a,z}$ generated by ϕ and ρ networks trained with 3 CF data. Each heatmap gives the prediction of $f_{a,z}$ of a vehicle in different horizontal and vertical (global) positions, and the (global) position of neighboring vehicles are represented by blue stars. A more detailed explanation is in Sec. V-D.

Fig. 2 depicts an example of the swapping task for three quadrotors (showing two out of the six swaps), which corresponds to column 2 of Table I. We observe that: i) when trained on at least three quadrotors, our approach significantly outperforms the baseline controller; and ii) the performance degrades significantly when only trained on two quadrotors, since the training data does not include data on superpositions.

D. Learned Neural Network Visualization

Fig. 3 depicts the prediction of $f_{a,z}$, trained with flying data of 3 multirotors. The color encodes the magnitude of $\hat{f}_{a,z}$ for a single multirotor positioned at different global (y, z) coordinates. The blue stars indicate the (global) coordinates of neighboring multirotors. All quadrotors are in the same x -plane. For example, in Fig. 3(c) there are two quadrotors hovering at $(0, -0.06, 0)$ and $(0, 0.06, 0)$. If we place a third quadrotor at $(0, -0.3, -0.5)$, it would estimate $\hat{f}_{a,z} = 4$ g as indicated by the white color in that part of the heatmap. All quadrotors are assumed to be stationary except for Fig. 3(b), where the one neighbor is moving at 0.8 m/s.

We observe that the interaction between quadrotors is non-stationary and sensitive to relative velocity, as well as not a simple superposition between pairs. In Fig. 3(b), the vehicle's neighbor is moving, and the prediction becomes significantly different from Fig. 3(a), where the neighbor is just hovering. Moreover, in Fig. 3(b) there is an interesting region with relatively large positive $\hat{f}_{a,z}$, which is consistent with our observations in flight experiments. We can also observe that the interactions are not a simple superposition of different pairs. For instance, Fig. 3(e) shows a significantly stronger updraft effect outside the downwash region than expected from a simple superposition of the prediction in Fig. 3(a).

Our approach can generalize well using data for 3 vehicles to a larger 4-vehicle system. In Fig. 3, all the predictions are from ϕ and ρ networks trained with 3 CF data, but

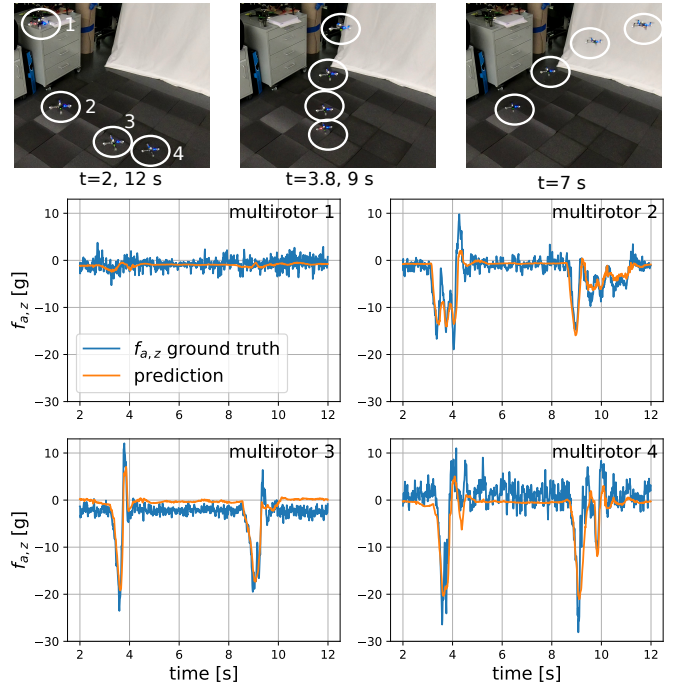


Fig. 4. Ground truth and $\hat{f}_{a,z}$ for a 4-vehicle swapping task. $\hat{f}_{a,z}$ is generated by neural networks trained with data from three vehicles. Our method generalizes well and predicts $f_{a,z}$ with high accuracy.

predictions for a 4-vehicle team (as shown in Fig. 3(e-f)) are still reasonable and work well in real flight tests (see Table I and Fig. 2). For this 4 CF swapping task, we compare ground truth $f_{a,z}$ and its prediction in Fig. 4. As before, the prediction is computed using neural networks trained with 3 CF flying data. We found that 1) for multirotor 3 and 4, $f_{a,z}$ is so high such that we cannot fully compensate it within our thrust limits; and 2) the prediction matches the ground truth very well, even for complex interactions (e.g., multirotor 2 in Fig. 4), which indicates that our approach generalizes well.

VI. CONCLUSION

In this paper, we present a decentralized controller that enables close-proximity flight of homogeneous multirotor teams. Our solution, *Neural-Swarm*, uses deep neural networks to learn the interaction forces between multiple quadrotors and only relies on relative positions and velocities of neighboring vehicles. We demonstrate in flight tests that our training method generalizes well to a varying number of neighbors, is computationally efficient, and reduces the worst-case height error by a factor of two or better. To our knowledge, our solution is the first that models interactions between more than two multirotors.

There are many directions for future work. First, one can extend our work to heterogeneous swarms, which may require extending the neural net architecture beyond spectral normalized deep sets. Second, one can use the learned interaction forces for motion planning and control of dynamically changing formations. Third, one can learn τ_a as well as $\mathbf{f}_a$ to improve the flight performance during aggressive maneuvers even further.

REFERENCES

- [1] S.-J. Chung, A. A. Paranjape, P. M. Dames, S. Shen, and V. Kumar, "A survey on aerial swarm robotics," *IEEE Transactions on Robotics (T-RO)*, vol. 34, no. 4, pp. 837–855, 2018. [Online]. Available: <https://doi.org/10.1109/TRO.2018.2857475>
- [2] D. Morgan, G. P. Subramanian, S.-J. Chung, and F. Y. Hadaegh, "Swarm assignment and trajectory optimization using variable-swarm, distributed auction assignment and sequential convex programming," *International Journal of Robotics Research (IJRR)*, vol. 35, no. 10, pp. 1261–1285, 2016. [Online]. Available: <https://doi.org/10.1177/0278364916632065>
- [3] X. Kan, J. Thomas, H. Teng, H. G. Tanner, V. Kumar, and K. Karydis, "Analysis of ground effect for small-scale uavs in forward flight," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3860–3867, Oct 2019. [Online]. Available: <https://doi.org/10.1109/LRA.2019.2929993>
- [4] W. Hönig, J. A. Preiss, T. K. S. Kumar, G. S. Sukhatme, and N. Ayanian, "Trajectory planning for quadrotor swarms," *IEEE Trans. Robotics*, vol. 34, no. 4, pp. 856–869, 2018. [Online]. Available: <https://doi.org/10.1109/TRO.2018.2853613>
- [5] J. van den Berg, S. J. Guy, M. C. Lin, and D. Manocha, "Reciprocal n -body collision avoidance," in *International Symposium on Robotics Research (ISRR)*, vol. 70, 2009, pp. 3–19. [Online]. Available: https://doi.org/10.1007/978-3-642-19457-3_1
- [6] P. L. Bartlett, D. J. Foster, and M. Telgarsky, "Spectrally-normalized margin bounds for neural networks," in *Conference on Neural Information Processing Systems (NIPS)*, 2017, pp. 6240–6249. [Online]. Available: <http://papers.nips.cc/paper/7204-spectrally-normalized-margin-bounds-for-neural-networks>
- [7] G. Shi, X. Shi, M. O'Connell, R. Yu, K. Aizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, "Neural Lander: Stable drone landing control using learned dynamics," in *International Conference on Robotics and Automation (ICRA)*, 2019, pp. 9784–9790. [Online]. Available: <https://doi.org/10.1109/ICRA.2019.8794351>
- [8] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Conference on Neural Information Processing Systems (NIPS)*, 2017, pp. 3391–3401. [Online]. Available: <http://papers.nips.cc/paper/6931-deep-sets>
- [9] H. M. Le, A. Kang, Y. Yue, and P. Carr, "Smooth imitation learning for online sequence prediction," in *International Conference on Machine Learning (ICML)*, vol. 48, 2016, pp. 680–688. [Online]. Available: <http://proceedings.mlr.press/v48/le16.html>
- [10] A. J. Taylor, V. D. Dorobantu, H. M. Le, Y. Yue, and A. D. Ames, "Episodic learning with control lyapunov functions for uncertain robotic systems," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 6878–6884. [Online]. Available: <https://doi.org/10.1109/IROS40897.2019.8967820>
- [11] R. Cheng, A. Verma, G. Orosz, S. Chaudhuri, Y. Yue, and J. Burdick, "Control regularization for reduced variance reinforcement learning," in *International Conference on Machine Learning (ICML)*, 2019, pp. 1141–1150. [Online]. Available: <http://proceedings.mlr.press/v97/cheng19a.html>
- [12] C. D. McKinnon and A. P. Schoellig, "Learn fast, forget slow: Safe predictive learning control for systems with unknown and changing dynamics performing repetitive tasks," *IEEE Robotics and Automation Letters (RA-L)*, vol. 4, no. 2, pp. 2180–2187, 2019. [Online]. Available: <https://doi.org/10.1109/LRA.2019.2901638>
- [13] M. Saveriano, Y. Yin, P. Falco, and D. Lee, "Data-efficient control policy search using residual dynamics learning," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 4709–4715. [Online]. Available: <https://doi.org/10.1109/IROS.2017.8206343>
- [14] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojea, E. Solowjow, and S. Levine, "Residual reinforcement learning for robot control," in *International Conference on Robotics and Automation (ICRA)*, 2019, pp. 6023–6029. [Online]. Available: <https://doi.org/10.1109/ICRA.2019.8794127>
- [15] I. Cheeseman and W. Bennett, "The effect of ground on a helicopter rotor in forward flight," *Aeronautical Research Council Reports And Memoranda*, 1955.
- [16] D. Yeo, E. Shrestha, D. A. Paley, and E. M. Atkins, "An empirical model of rotorcraft uav downwash for disturbance localization and avoidance," in *AIAA Atmospheric Flight Mechanics Conference*, 2015. [Online]. Available: <https://arc.aiaa.org/doi/abs/10.2514/6.2015-1685>
- [17] D. Shukla and N. Komerath, "Multirotor drone aerodynamic interaction investigation," *Drones*, vol. 2, no. 4, 2018. [Online]. Available: <https://www.mdpi.com/2504-446X/2/4/43>
- [18] K. P. Jain, T. Fortmuller, J. Byun, S. A. Mkihharju, and M. W. Mueller, "Modeling of aerodynamic disturbances for proximity flight of multirotors," in *2019 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2019, pp. 1261–1269. [Online]. Available: <https://doi.org/10.1109/ICUAS.2019.8798116>
- [19] D. Morgan, S.-J. Chung, and F. Y. Hadaegh, "Model predictive control of swarms of spacecraft using sequential convex programming," *Journal of Guidance, Control, and Dynamics*, vol. 37, no. 6, pp. 1725–1740, 2014.
- [20] M. DeBord, W. Hönig, and N. Ayanian, "Trajectory planning for heterogeneous robot teams," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 7924–7931. [Online]. Available: <https://doi.org/10.1109/IROS.2018.8593876>
- [21] D. Mellinger, A. Kushleyev, and V. Kumar, "Mixed-integer quadratic program trajectory generation for heterogeneous quadrotor teams," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2012, pp. 477–483. [Online]. Available: <https://doi.org/10.1109/ICRA.2012.6225009>
- [22] A. Liu, G. Shi, S.-J. Chung, A. Anandkumar, and Y. Yue, "Robust regression for safe exploration in control," *CoRR*, vol. abs/1906.05819, 2019. [Online]. Available: <http://arxiv.org/abs/1906.05819>
- [23] S. Bandyopadhyay, S.-J. Chung, and F. Y. Hadaegh, "Nonlinear attitude control of spacecraft with a large captured object," *Journal of Guidance, Control, and Dynamics*, vol. 39, no. 4, pp. 754–769, 2016.
- [24] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in *IEEE Conference on Decision and Control (CDC)*, 2010, pp. 5420–5425. [Online]. Available: <https://doi.org/10.1109/CDC.2010.5717652>
- [25] S.-J. Chung, S. Bandyopadhyay, I. Chang, and F. Y. Hadaegh, "Phase synchronization control of complex networks of lagrangian systems on adaptive digraphs," *Automatica*, vol. 49, no. 5, pp. 1148–1161, 2013.
- [26] H. Khalil, *Nonlinear Systems*, ser. Pearson Education. Prentice Hall, 2002.
- [27] J. A. Preiss, W. Hönig, G. S. Sukhatme, and N. Ayanian, "Crazyswarm: A large nano-quadcopter swarm," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 3299–3304. [Online]. Available: <https://doi.org/10.1109/ICRA.2017.7989376>
- [28] Bitcraze. (2015) Crazyflie 2.0 thrust investigation. [Online]. Available: <https://wiki.bitcraze.io/misc:investigations:thrust>
- [29] J. Förster, "System identification of the crazyflie 2.0 nano quadcopter," Master's thesis, ETH Zurich, Zurich, 2015.
- [30] M. Faessler, D. Falanga, and D. Scaramuzza, "Thrust mixing, saturation, and body-rate control for accurate aggressive quadrotor flight," *IEEE Robotics and Automation Letters (RA-L)*, vol. 2, no. 2, pp. 476–482, 2017. [Online]. Available: <https://doi.org/10.1109/LRA.2016.2640362>
- [31] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," in *IEEE International Conference on Robotics and Automation (ICRA)*, 1985, pp. 500–505. [Online]. Available: <https://doi.org/10.1109/ROBOT.1985.1087247>
- [32] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Conference on Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024–8035. [Online]. Available: <http://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [33] A. Molchanov, T. Chen, W. Hönig, J. A. Preiss, N. Ayanian, and G. S. Sukhatme, "Sim-to-(multi)-real: Transfer of low-level robust control policies to multiple quadrotors," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 59–66. [Online]. Available: <https://doi.org/10.1109/IROS40897.2019.8967695>